Power Monitoring and Energy Estimate

Name: Pooja Oswal
Project: Machine Power Monitoring
Language: Python
Database: MySQL 8.0

1. Problem Understanding

The task is to monitor the power readings of different machines and check whether they exceed their power limits. The Python program calculates the energy used by each reading and gives a summary for every machine. The SQL queries are used to find daily power statistics and readings that exceed the power limit.

2. Assumptions

1. Each reading represents a time interval of 15 minutes.
2. Energy is calculated using the formula: Power × 0.25 hours
3. A reading is HIGH only when its power is greater than the machine's power limit. Otherwise, it is NORMAL.
4. Readings with unknown machine IDs are skipped
5. Missing, non-numeric, infinite, or negative power values are treated as invalid.
6. Power is measured in kW and energy in kWh.
7. Zero power is valid and gives zero energy.

3. Algorithm

1. Store the power limits for each machine.
2. Take the machine readings as input.
3. Check whether the machine ID exists and the power value is valid.
4. Skip readings with unknown machine IDs or invalid power values.
5. Compare each valid power reading with its machine's limit.
6. Assign HIGH or NORMAL status based on the comparison.
7. Calculate energy using Power × 0.25.
8. Update the total energy, average power, peak power, and HIGH reading count for each machine.
9. Display the results and final summary.
10. Execute the SQL queries to calculate daily statistics and find readings above the configured power limits.

4. Logic Used

Python Logic

A `for` loop is used to process each reading. Conditions are used to validate the input and compare power readings with their limits.

A dictionary stores the summary for each machine. The `max()` function is used to find peak power, and the average power is calculated by dividing total power by the number of valid readings.

Energy formula:

Energy (kWh) = Power (kW) × Time (hours)

Since each reading represents 15 minutes:

Energy = Power × 0.25

SQL Logic

Query 1: Daily Machine Summary

The first query joins the `Machines` and `SensorReadings` tables using `machine_id`.

- `AVG()` calculates average power.
- `MAX()` finds maximum power.
- `CASE WHEN` counts readings above the configured limit.
- `SUM()` calculates total estimated energy.
- `GROUP BY` groups the results by machine and date.

Query 2: Readings Above Power Limit

The second query joins the same tables and uses a `WHERE` condition to select readings where measured power exceeds the machine's limit.

The excess power is calculated by subtracting the configured limit from the measured power. `ORDER BY` sorts the results from highest excess power to lowest.

5. Testing

I tested the Python program with the original readings and additional test cases.

- **Normal reading:** The power is below the limit.
- **HIGH reading:** The power exceeds the limit.
- **Unknown machine:** The program displays a message and skips the reading.
- **Zero power:** The reading is accepted and energy is zero.

Test Case	Input	Expected Behaviour
1. Normal reading	Machine 101, power 16.5 kW	Status is NORMAL because 16.5 is below the 18 kW limit.
2. HIGH reading	Machine 101, power 19.1 kW	Status is HIGH because 19.1 exceeds the 18 kW limit.
3. Unknown machine	Machine 999, power 15 kW	Display a message and skip the reading.
4. Zero power	Machine 101, power 0 kW	Accept the reading, with zero energy consumption.

6. Complexity

Let `n` be the number of input readings and `m` be the number of machines in the final summary.

The program processes each reading once, so the main processing takes **O(n)** average time. Sorting the machine summaries takes **O(m log m)** time.

Therefore, the overall time complexity is **O(n + m log m)**.

The program stores a summary for each machine, so the summary requires **O(m)** space. The input readings themselves are stored separately.